

Semantic-Aware Advanced Persistent Threat Detection Using Autoencoders on LLM-Encoded System Logs

Waleed Khan Mohammed,¹ Nouar AlDahoul,² Hezerul Abdul Karim,^{3, 1, a)}
Zahirul Arief Irfan Bin Shahrul Anuar,¹ and Mousa Sufian Mousa Mitani¹

¹*Faculty of Artificial Intelligence and Engineering, Multimedia University, Persiaran Multimedia, Cyberjaya, 63100, Selangor, Malaysia*

²*Computer Science Division, New York University, Abu Dhabi, UAE*

³*Centre for Image and Vision Computing, COE for Artificial Intelligence*

^{a)}Corresponding author: hezerul@mmu.edu.my

Abstract. Advanced Persistent Threats (APTs) utilize "low-and-slow" tactics to evade standard security measures, making them exceptionally difficult to detect. Traditional defense mechanisms, relying heavily on statistical anomalies or shallow machine learning, often lack the depth to identify these sophisticated campaigns. Similarly, while provenance-based techniques track causal history, they frequently overlook the rich semantic context within system logs required to distinguish benign irregularities from malicious intent. To address this gap, this paper proposes MPNet-AE, a novel anomaly detection framework that leverages Large Language Models (LLMs) to extract semantic meaning from unstructured logs. The method converts raw system events into high-dimensional embeddings using a pre-trained transformer. An Autoencoder (AE) is then trained on these embeddings to learn the manifold of normal behavior, identifying attacks as statistical outliers with high reconstruction errors. This approach effectively captures non-linear relationships that simpler models often miss. This paper evaluated the model against realistic APT scenarios from the DARPA Transparent Computing (TC) dataset. Comparative analysis shows that MPNet-AE consistently outperforms standard unsupervised baselines, including Isolation Forest, One-Class SVM, and PCA. Notably, MPNet-AE achieved the highest detection accuracy with an average AUC-ROC of 0.95 across diverse datasets. While linear methods like PCA performed competitively, our semantic approach proved significantly more robust than density-based models in complex threat environments. These findings demonstrate that integrating LLM-driven semantic understanding with reconstruction-based learning offers a superior defense against the stealthy behaviors characteristic of modern APT campaigns.

INTRODUCTION

APTs, considered a major threat to information security, differ from common malware attacks in their stealth, rarity, and extended attacks conducted by sophisticated attackers [1, 2]. Attackers often resort to "living off the land" to persist on the network without getting detected, thereby using and further extending system tools such as PowerShell and Windows Management Instrumentation (WMI) to carry out their malicious tasks seamlessly, much like their normal administrative tasks. The result is that system logs have become a major source of forensics used to analyze and identify attacks. The raw amounts of system-generated logs continue to make it impossible to scan them to identify any discrepancies, thus causing the need to use automated tools to identify anomalies [3].

Although log analysis is of utmost importance, there are some major challenges for traditional anomaly identification techniques. Traditional techniques like IForest [4], One-OC-SVM [5] and PCA [6] depended mainly on the statistical properties or the token frequency. These "shallow" learning methods regard log entries as points in a data set or merely as a bag-of-words, without regard to the extensive semantic context contained in the messages issued by the system [7, 8, 9]. For example, a statistical model may have difficulty distinguishing a legal password reset operation by a system administrator from a privileged escalation attack initiated by a hacker if the statistical probability of the words is relatively close in value.

In order to overcome these drawbacks, this research paper proposes a transition from the conventional statistical anomaly detection method to semantic anomaly detection. The approach of utilizing models such as `all-mpnet-base-v2` [10] aims not only to enhance generalization by creating high-dimensional semantic embeddings of raw, unstructured system logs, but also to improve pattern recognition, enabling subtler behavioral differences to be identified more effectively. Later, semantic embeddings are fed into deep reconstruction models, particularly autoencoders [11, 12], to robustly capture normal system behavior. Reconstruction loss is used to track anomalies. Events that do not get a good reconstruction by the model are considered to imply some intrusion. This method can discover malicious activities not because they are infrequent, but because their semantic behavior is different from the normal one.

The paper contributions are as follows:

1. This work builds on previous research [13] by proposing an unsupervised anomaly detection framework that used a pre-trained transformer language models, such as `all-mpnet-base-v2`, instead of the earlier Bidirectional Encoder Representations from Transformers (BERT) variants [14], combined with autoencoders to detect APTs through semantic differences.
2. This work performs a comparative evaluation against three known baselines such as IForest, OC-SVM, and PCA. The results show the benefits of applying a pre-trained embedding model to complex log data.
3. This work utilized data from the DARPA TC project [1, 15]. The experimental results show that the combination of `all-mpnet-base-v2` and autoencoder achieves higher AUC-ROC scores than those of the traditional baselines.

RELATED WORK

Recent cybersecurity research advances have increasingly explored the integration of deep learning due to the limitations of traditional anomaly detection and signature-based defense mechanisms [16, 17, 18]. Specifically, semantic understanding, cross-domain correlation, and contextual reasoning have become the main enablers for detecting stealthy and low-frequency APTs. One of the works leverages LLMs and unsupervised modeling to enhance APT detection, correlation, and investigation [13, 19]. They present the development of a semantic anomaly detection framework that includes a system provenance traces transformed into natural language and then embedded into dense vectors by transformer models like BERT, ALBERT, and MiniLM, followed by an autoencoder-based reconstruction for anomaly scoring. Their proposed method outperformed traditional baselines. However, their proposed framework underlines several challenges like computational overhead and real-time applicability.

In [20], the authors introduce a new modular framework where different agents handle specific tasks, such as checking emails, analyzing logs, and scanning IP addresses. The framework also includes a result correlation module that combines outputs from these agents to detect multi-step attacks. By using LLaMA-3.3-70B [21] along with methods like retrieval-based generation and chain-of-thought prompting, the approach shows strong performance in producing meaningful, domain-level insights and effectively identifying potential security threats [20].

In other works, authors propose a two-phased strategy integrating anomaly detection using graph structures with LLM-assisted analysis for constructing comprehensive APT attack scenarios from audit logs [22]. The proposed model uses One-Class Relational Graph Convolutional Networks for anomaly detection within subgraphs. The detection is based on both structural and behavioral characteristics. An LLM-assisted retrieval pipeline is then employed to construct error-free, human-readable attack stories that are consistent with the APT kill-chain model. In their experiments with the DARPA TC3, OpTC, and NODLINK corpora, they found that there is immense improvement over existing solutions for both accuracy and investigation quality. However, there is still a need for future focus on simulating concurrency attacks, generalization, and integration with traffic data.

METHODOLOGY

In this section, the datasets used in this work is described. In addition, proposed improvement over existing methods [13] is discussed, namely the MPNet-AE APT detection framework.

Datasets

The experimental evaluation is performed on the basis of the DARPA TC datasets [15], which form a widely adopted set of benchmarks for APT detection research. We used 5DIR, CADETS, CLEARSCOPE, THEIA, and TRACE scenarios, each capturing fine-grained system-level activities across heterogeneous operating systems and attack campaigns [13]. These datasets contain fine-grained provenance information, including process executions, system calls, file accesses, and network communications, providing a realistic trace of enterprise-scale system behavior [15].

All datasets are ingested and normalized using the Automatic Detection of Advanced Persistent Threats (ADAPT) project’s ingester, which provides a consistent schema representation across various scenarios [13]. To analyze the contribution of different behavioral dimensions, each dataset is decomposed into five distinct provenance contexts:

TABLE 1. Experimental datasets of DARPA’s TC program used in our study. A dataset entry (columns 3 to 7) is described by the number of rows (processes) / the number of columns (attributes). For instance, with ProcessAll (PA) obtained from the second scenario of Linux, the dataset comprises 279,369 rows and 457 attributes, including 100 APT attacks (0.036%).

	Size	<i>PE</i>	<i>PX</i>	<i>PP</i>	<i>PN</i>	<i>PA</i>	<i>nb_attacks</i>	$\% \frac{nb_attacks}{nb_processes}$
BSD	265 MB	224624 / 32	224146 / 136	223780 / 38	42888 / 63	224624 / 266	11	0.005
Windows	36.0 MB	11151 / 31	11077 / 372	10992 / 81	329 / 126	11151 / 607	9	0.081
Linux (1)	3.63 GB	282087 / 26	271088 / 141	263730 / 46	6580 / 6226	282103 / 6436	46	0.016
Linux (2)	465 MB	270652 / 19	278363 / 160	279090 / 71	48048 / 210	279369 / 457	100	0.036
Android	18.1 MB	12106 / 28	12106 / 45	24 / 12	1116 / 214	12106 / 296	13	0.107

- ProcessEvent (PE): events at the system and kernel level.
- ProcessExec (PX): it runs binaries and traces of commands and shows all the processes currently running at this point.
- ProcessParent (PP): it refers to parent-child process relationships.
- ProcessNetflow (PN): this module represents network communication behavior.
- ProcessAll (PA): a single and uniform view of all contexts.

This multiview decomposition allows the anomaly detection performance to be evaluated in fine-grained terms with respect to various system attributes. The table 1 summarizes the dataset by illustrating its highly imbalanced nature with APT attacks.

One of the biggest challenges in the DARPA TC datasets is the fact that they are highly imbalanced, meaning that less than 0.004% of the records are malicious APT actions among APT networks. This makes the datasets ideal for testing models that are semantic-aware and reconstruction-based.

Method

The MPNet-AE APT detection framework receives process and network activity data from the provenance database and converts them into semantic embeddings generated by a pre-trained language model `all-mpnet-base-v2` [23], followed by an autoencoder trained for anomaly detection.

Data Provenance Graphs

The proposed framework relies on a provenance database as the core component for recording system execution behavior at the process level [1, 15]. A single record of this database corresponds to an activity of an individual process. It consists of metadata, such as the process identifier and the sequence of system events generated during its execution. Such events can include system calls, executed binaries, relationships among processes considering their creation, which creates parent-child links, and active network interactions [1, 15].

Taken together, these logs provide a summarize view of system and network behaviour over time. Rather than examining raw log entries in isolation, the provenance-based representation captures contextual relationships between activities that can be leveraged to carry out more meaningful behaviour analysis. To facilitate semantic learning, each provenance record is later transformed into a structured textual description suitable for processing by a language model.

Textual Conversion and Embedding Generation

To facilitate the semantic interpretation of the system behavior, every record of the provenance is generated in the format of a natural language description, capturing the observed activities of the process. This helps fill the gap

between the system logs and a human-interpretable description of the behavior. The sentence generation technique adopted follows the approach generally used in semantic log representation and APT detection techniques.

Features will only be listed for each process if they are active. The system will depend on the contextual view, whether it is process events, binary execution, parent processes, network traffic, or all contexts combined, in order to construct meaningful sentences to describe process activities, their interaction with the system, and whether they communicated with the network.

This sentence construction, which takes into account the context, ensures that the sentences or text generated are more likely to be similar to those written by a human and are not just a list of keywords or representations based on frequency.

To overcome the limitations of token-based and manually designed log features, we used `all-mpnet-base-v2` model [10, 23], a sentence embedding model based on Microsoft MPNet. MPNet is a transformer pretrained with a combination of Masked Language Modeling (MLM) and Permuted Language Modeling (PLM). It inherits the advantages of BERT and XLNet and avoids their limitations. `all-mpnet-base-v2` yields richer contextual representations than MLM-only BERT models. Combined with sentence-level contrastive fine-tuning, this enables the generation of high-quality semantic sentence embeddings.

`all-mpnet-base-v2` model [10, 23] was already fine-tuned on a dataset of 1B sentence pairs. The Training process used a contrastive learning objective by giving one sentence, the model learns to identify its true paired sentence from a set of randomly sampled alternatives [10, 23]. The textual descriptions created previously are coded in the form of 768 dimensional dense vectors using `all-mpnet-base-v2`. These vectors are further normalized to ensure stability during autoencoder training.

These semantic embeddings are the key input features to the anomaly detection model. Unlike manually engineered features, the embeddings contain high-level, behavioral semantic meanings, enabling the system to identify subtle anomalies that share similar surface-level patterns with normal behaviors.

Autoencoder

The autoencoder consists of two major pieces: the encoder and the decoder. The encoder takes the high-dimensional semantic embeddings and maps them to a lower-dimensional space, which learns the essential characteristics of normal behavior in this process. It then decodes this mapping to reproduce the semantic embedding [11, 13]. When the AE is trained, the error in reconstruction for normal instances is minimized. In turn, this implies that the AE performs extremely well in reconstructing benign instances and not so well in reconstructing malicious instances. This is the starting point for implementing the Anomaly detector.

To replicate the normal system behavior, a vanilla autoencoder [11] is used in this work. This autoencoder works by replicating the embeddings, which helps it to understand the patterns from the normal system processes [13]. This process was done using the unlabeled malicious data.

Experimental Setup

The aim of the set of experiments carried out in this work is to investigate the efficiency of the MPNet-AE framework. The autoencoder was trained using the embeddings from the typical system processes not the malicious ones. The trained autoencoder model was tested on a hybrid dataset that has both normal and anomalous data. To prevent data leakage and also produce a realistic scenario, the data was partitioned sequentially. For example, the first 80% of the benign (normal) process records were used only for training and validating the autoencoder. The remaining 20% of benign records and 100% of all anomalous APT records were combined to form the test set. This way, the model is trained against a historical baseline and is evaluated on only future events that it has never seen before.

The model's performance was evaluated in terms of reconstruction error analysis and AUC-ROC scores to determine how well the model can separate normal activities from the attacks based on a certain threshold as done in previous works [13].

A reconstruction error, which was obtained after training an autoencoder, was employed to determine the anomaly score for each sample. A sample with a high reconstruction error will be more prone to anomalies, hence classified as an anomaly. The classification boundary based on the value of the reconstruction error, determines whether the data points fall into the normal or anomalous class. This boundary was chosen based on the validation dataset.

The experiments involving MPNet embeddings and the autoencoder were run on an NVIDIA RTX 1080 Ti GPU with 11 GB of VRAM. Other baseline models, such as Isolation Forest, OC-SVM, and PCA, were run on the CPU.

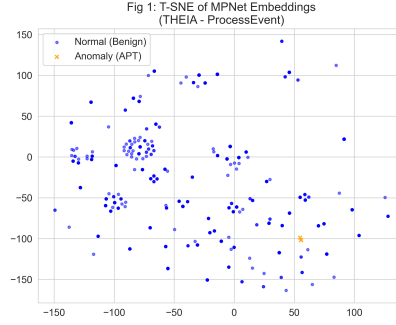


FIGURE 1. T-SNE visualizations of embeddings using `all-mpnet-base-v2`. Blue points (label 0) represent the normal data, whereas the orange crosses (label 1) represent the anomalies. In this example, data is represented for the PE dataset of the Linux OS.

RESULTS AND DISCUSSION

In this section, the results of experiments performed to detect anomalous attacks are presented and discussed.

Semantic Embedding Strategy using MPNet

The authors visualized the embedding of two attacks' classes using t-distributed Stochastic Neighbor Embedding (T-SNE) [24] for qualitative analysis.

As illustrated in Figure 1, normal process activities form compact clusters, while anomalous samples appear as isolated points, indicating effective semantic discrimination. The observed patterns of clustering closely coincide with the high AUC-ROC of the autoencoder, which supports the design principles of the MPNet-AE framework.

The raw logs can be transformed into text by first filtering and extracting the active attributes of the system (e.g., Process IDs, IPs, File Paths) and filtering out empty fields to reduce noise. The active attributes are then mapped to predefined syntactic templates that are typically associated with the event types and concatenated together with connection words (e.g., and, but, etc.) in chronological order to form a single trace that would be clear and readable by humans. For example, you could turn the above example into a single trace as follows:

'Process 1054 started /bin/bash and connected to socket 192.168.1.5:80 and modified /etc/passwd'.

These sequences are converted into tokens and processed by MPNet to produce 768-dimensional embeddings. Token representations are averaged using mean pooling to create a single fixed-size vector for each process instance. This method groups similar benign behaviors closely in the embedding space, while unseen or malicious APT activities are placed farther away, making anomalies easier to distinguish.

Autoencoder Training and Detection Mechanism

We fed the semantic representations into an autoencoder designed to act as a normal behavior model. The autoencoder uses a symmetric compression structure that gradually reduces dimensionality, allowing it to learn a compact representation of normal activity patterns.

The model was fitted only on embeddings of normal behaviors using mean squared error (MSE) as the reconstruction loss metric. The number of epochs used in training was 15, training and validation loss curves had a rapid convergence at the beginning of the epochs and then reached a minimum. The similarity between the training and validation losses points to the strong generalization and to the insignificant overfitting. Figure 2 shows the training and validation MSE curves over the epochs, indicating stable convergence and minimal overfitting.

When normal and abnormal samples combined in the testing time, the reconstruction error clearly separates them. Benign activities produced low reconstruction errors, while anomalous APT-related activities generated much higher errors because they differ from the learned patterns. This makes anomalies easy to detect using a threshold. As shown in Figure 3, most normal samples have low reconstruction error, whereas anomalous activities exceed the threshold and are correctly identified.

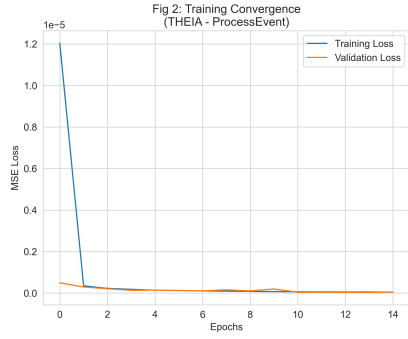


FIGURE 2. Training and validation loss curve for the autoencoder.

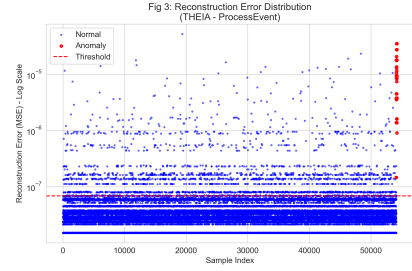


FIGURE 3. Scatter plot of the reconstruction errors in autoencoder for each sample. The red dashed line shows the anomaly detection threshold.

Performance Analysis

We compared the proposed MPNet-AE framework with three popular unsupervised anomaly detection baselines such as IForest, OC-SVM, and PCA using the AUC-ROC, which is quite effective in highly skewed data. As shown in Figure 5, MPNet-AE consistently achieves higher AUC scores, demonstrating strong generalization across heterogeneous environments.

In all the datasets and provenance settings, MPNet-AE is always seen to be performing better than the traditional baselines. Specifically, it can reach AUC scores higher than 0.95 in 11 out of 23 experimental conditions as shown in Figure 5. The ROC curve is shown in Figure 4, where MPNet-AE achieves higher true positive rates across all thresholds, confirming its robustness under extreme class imbalance.

Since MPNet is a pre-trained model, no training was performed. On the contrary, the autoencoder requires end-to-end training because it has as a symmetric feed-forward neural network operating on 768-dimensional MPNet sentence embeddings. The encoder consists of two fully connected layers that progressively compress the input from 768 to 512 and then to a 128-dimensional latent representation, using ReLU activations to introduce non-linearity. The decoder mirrors this structure, reconstructing the input by expanding the latent vector through a 512-unit hidden layer back to the original 768-dimensional space. The model was trained using MSE reconstruction loss and optimized with the Adam optimizer at a learning rate of 0.001. Training was performed for 15 epochs with a batch size of 128.

For the OC-SVM model, the parameter ν was set to 0.01, indicating that approximately 99% of the training data is assumed to be normal, while up to 1% may be treated as anomalies. This choice makes the model conservative in labeling anomalies, focusing on learning a tight boundary around normal data. The radial basis function (RBF) kernel was used to capture non-linear patterns in the data, allowing the model to effectively separate normal behavior from outliers in complex feature spaces. Additionally, γ was set to auto, meaning it is automatically determined based on the number of features, which helps balance model flexibility and generalization without manual tuning.

For the Isolation Forest model, the number of estimators was set to 100, meaning the algorithm builds 100 isolation trees. Each tree used binary splits, resulting in two branches per node, which is intrinsic to the isolation mechanism. The tree depth was not fixed manually. Instead, it was automatically determined by the `max_samples` parameter which was set to auto, each tree was trained on a random subset of samples.

For PCA, we set $n_components = 0.95$ means that PCA is configured to retain 95% of the total variance in the original data rather than a fixed number of components. The algorithm automatically selected the minimum number of principal components required to explain at least 95% of the variance, effectively reducing dimensionality while preserving most of the information.

The autoencoder phase of the MPNet-AE framework showed strong performance with low computing requirements and good real-world scalability. Testing was done using an NVIDIA RTX 1080 Ti with 11GB VRAM. For very large datasets such as THEIA and TRACE, the system processed hundreds of thousands of samples in about 65 seconds. This equals more than 4,000 logs per second, or 0.23 milliseconds per log. For the smaller CADETS dataset, processing took around 52 seconds. Even smaller datasets like 5DIR and CLEARSCOPE finished in less than 4 seconds. The most demanding step was creating the first dense embeddings with the MPNet transformer, mainly because of GPU memory limits. In comparison, the autoencoder reconstruction stage was very lightweight and fast. To handle the high-speed log streams used in enterprise environments, the framework can be scaled using stream-

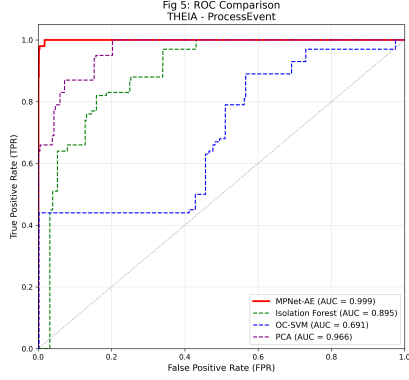


FIGURE 4. ROC curve for the MPNet-AE model.

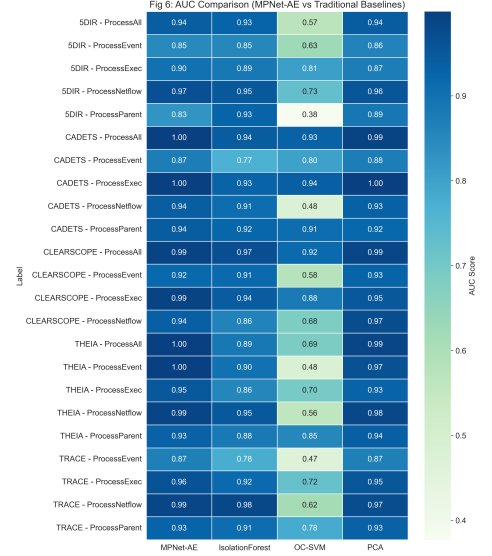


FIGURE 5. AUC heatmap comparing the performance of MPNet-AE, IForest, OC-SVM, and PCA across datasets from multiple OS (Windows, Linux, Android, and BSD).

processing systems and newer hardware. For example, upgrading to a stronger GPU would improve embedding speed and support larger batch sizes and more parallel processing. This would help deliver real-time threat detection in live Security Operations Centers (SOCs).

The suggested solution provides a close to perfect detection rate in complicated cases like THEIA-ProcessEvent, which is significantly higher than isolated forest and the OC-SVM as shown in Figure 5. Similarly, for TRACE, ProcessNetflow, with which patterns in the network traffic are very dynamic and noisy, MPNet-AE still has a high detection rate, but OC-SVM suffers from a drastic performance drop. This finding underlines the lack of effectiveness of traditional methods in handling high-dimensional semantic embeddings and nonlinear manifolds.

We found that PCA is competitive in fairly organized data like CADETS. PCA struggles most relative to MPNet-AE on complex relational views, especially ProcessExec and ProcessNetflow across datasets. PCA is rarely catastrophic, but it is systematically weaker than MPNet-AE in several settings.

Overall, MPNet-AE produces significant improvement compared to classical unsupervised baselines, proving that semantic representation learning with reconstruction-based modeling has a decisive advantage in the context of detection with APT in the process.

CONCLUSION

This paper introduces MPNet-AE, an unsupervised and semantically aware anomaly detection system designed for APT detection in provenance data. The approach captures contextual and semantic relationships among low-level system logs, information that traditional statistical features cannot represent, by converting logs into structured natural language and encoding them using the MPNet language model. The resulting embeddings are analyzed using reconstruction errors from an autoencoder trained solely on benign data, making the system effective at detecting stealthy and low-frequency attacks.

Tests on multiple DARPA TC datasets show that MPNet-AE consistently outperforms traditional unsupervised methods, achieving high AUC-ROC scores across different provenance scenarios, even when class distributions are highly imbalanced. Specifically, our comparative analysis reveals that MPNet-AE achieves the highest average AUC-ROC (0.95) across all tested datasets. While linear projection methods like PCA performed competitively in some contexts, MPNet-AE demonstrated superior robustness compared to density-based baselines, significantly outperforming OC-SVM and IForest in complex scenarios such as CADETS and THEIA. Both quantitative results and visualizations confirm that malicious activities occupy distinct regions in the learned semantic space. This indicates that combining transformer-based representations with non-linear reconstruction learning is effective for detecting

APTs.

Future research directions involve fine-tuning language models using logs specific to cybersecurity to further improve semantic discrimination. It is also worth exploring temporal and graph-based dependencies among processes to better detect multi-stage APT campaigns. Furthermore, the authors aim to examine other reconstruction models and adaptive online learning strategies to improve resilience in dynamic real-world settings. In addition, the expansion of the framework to live enterprise systems with the mechanisms of explainability will make it more practical in security operations.

REFERENCES

1. G. Berrada, J. Cheney, S. Benabderrahmane, W. Maxwell, H. Mookherjee, A. Theriault, and R. Wright, "A baseline for unsupervised advanced persistent threat detection in system-level provenance," *Future Generation Computer Systems* **108**, 401–413 (2020).
2. P. Danso, L. McGevna, J. Chow, and J. Luo, "Llm-based anomaly detection for digital substation cybersecurity," (2025) pp. 128–133.
3. MITRE ATT&CK, "Command and scripting interpreter: Powershell," <https://attack.mitre.org/techniques/T1059/001/> (2024).
4. F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 eighth ieee international conference on data mining (IEEE, 2008)* pp. 413–422.
5. L. M. Manevitz and M. Yousef, "One-class svms for document classification," *Journal of machine Learning research* **2**, 139–154 (2001).
6. H. Abdi and L. J. Williams, "Principal component analysis," *Wiley interdisciplinary reviews: computational statistics* **2**, 433–459 (2010).
7. V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.* **41** (2009), 10.1145/1541880.1541882.
8. M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17* (Association for Computing Machinery, New York, NY, USA, 2017) p. 1285–1298.
9. Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)* (2016) pp. 102–111.
10. K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "Mpnnet: Masked and permuted pre-training for language understanding," *Advances in neural information processing systems* **33**, 16857–16867 (2020).
11. G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *science* **313**, 504–507 (2006).
12. M. A. Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, N. Tihanyi, T. Bisztray, and M. Debbah, "Generative ai in cybersecurity: A comprehensive review of llm applications and vulnerabilities," *Internet of Things and Cyber-Physical Systems* **5**, 1–46 (2025).
13. S. Benabderrahmane, P. Valtchev, J. Cheney, and T. Rahwan, "Apt-llm: Embedding-based anomaly detection of cyber advanced persistent threats using large language models," in *2025 13th International Symposium on Digital Forensics and Security (ISDFS)* (2025) pp. 1–6.
14. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)* (2019) pp. 4171–4186.
15. "Transparent computing engagement 3 data release," <https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md>.
16. N. AlDahoul, H. Abdul Karim, and A. S. Ba Wazir, "Model fusion of deep neural networks for anomaly detection," *Journal of Big Data* **8**, 106 (2021).
17. N. H. A. Mutalib, A. Q. M. Sabri, A. W. A. Wahab, E. R. M. F. Abdullah, and N. AlDahoul, "Explainable deep learning approach for advanced persistent threats (apts) detection in cybersecurity: A review," *Artificial Intelligence Review* **57**, 297 (2024).
18. N. H. A. Mutalib, A. Q. M. Sabri, A. W. A. Wahab, E. R. M. F. Abdullah, and N. AlDahoul, "An explainable recursive feature elimination to detect advanced persistent threats using random forest classifier," *arXiv preprint arXiv:2511.09603* (2025).
19. H. F. Atlam, "Llms in cyber security: Bridging practice and education," *Big Data and Cognitive Computing* **9**, 184 (2025).
20. Y. Hmimou, M. Tabaa, A. Khiat, and Z. Hidila, "A multi-agent system for cybersecurity threat detection and correlation using large language models," *IEEE Access* **13**, 150199–150215 (2025).
21. A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783* (2024).
22. A. Aly, E. Mansour, and A. Youssef, "Ocr-apt: Reconstructing apt stories from audit logs using subgraph anomaly detection and llms," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security* (2025) pp. 261–275.
23. "sentence-transformers/all-mpnet-base-v2," <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>.
24. L. v. d. Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research* **9**, 2579–2605 (2008).
25. A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Communications Surveys Tutorials* **21**, 1851–1877 (2019).
26. A. Ali and M. C. Ghanem, "Beyond detection: large language models and next-generation cybersecurity," *SHIFRA* **2025**, 81–97 (2025).
27. N. O. Jaffal, M. Alkhanafseh, and D. Mohaisen, "Large language models in cybersecurity: Applications, vulnerabilities, and defense techniques," *arXiv preprint arXiv:2507.13629* (2025).
28. S. Afnan, M. Sadia, S. Iqbal, and A. Iqbal, "Logshield: a transformer-based apt detection system leveraging self-attention," *arXiv preprint arXiv:2311.05733* (2023).
29. G. Berrada and J. Cheney, "Aggregating unsupervised provenance anomaly detectors," in *Proceedings of the 11th USENIX Conference on Theory and Practice of Provenance, TAPP'19* (USENIX Association, USA, 2019) p. 4.